Prática — Hash

DCC075 – Segurança em Sistemas Computacionais · UFJF · Lucas Gonçalves Rocha · Matrícula 202265187C · Individual · Outubro de 2026

1. SHA-256 passo a passo

O SHA-256 é uma função de hash que recebe uma mensagem de qualquer tamanho e devolve sempre 256 bits, representados por 64 dígitos hexadecimais. Ele é determinístico (a mesma mensagem gera sempre o mesmo hash) e de mão única (não é possível recuperar a mensagem a partir do hash). Como existem infinitas mensagens e um número finito de hashes, colisões existem, mas encontrar duas mensagens com o mesmo hash é computacionalmente inviável. Além disso, ele tem efeito avalanche: mudar um único caractere muda o hash inteiro. Por exemplo, “RedBlockBlue” gera f8f05f79…3bd9, enquanto “redblockblue” gera 77dc7473…a693.

Passo 1 – Conversão para binário. Cada caractere da mensagem é convertido em 8 bits, seguindo a tabela ASCII (UTF-8). A mensagem “RedBlockBlue” tem 12 caracteres, ou seja, 96 bits; o “R”, por exemplo, vira 01010010.

Passo 2 – Preenchimento (padding). O SHA-256 processa a mensagem em blocos de 512 bits, então ela precisa ser completada. Primeiro, acrescenta-se um bit 1; depois, zeros até o tamanho chegar a 448 bits (módulo 512); por fim, acrescenta-se o tamanho original da mensagem, escrito em 64 bits. No exemplo: 96 bits da mensagem + 1 bit “1” + 351 zeros + 64 bits com o valor 96 = 512 bits, exatamente um bloco. Gravar o tamanho no final garante que mensagens diferentes nunca fiquem iguais depois do preenchimento.

Passo 3 – Valores iniciais e constantes. São definidos 8 valores iniciais de 32 bits (H0 a H7), que vêm da parte fracionária das raízes quadradas dos 8 primeiros números primos (H0 = 6a09e667, por exemplo), e 64 constantes (K0 a K63), que vêm da parte fracionária das raízes cúbicas dos 64 primeiros primos. O uso de raízes de primos mostra que esses números não foram escolhidos para esconder uma fraqueza.

Passo 4 – Agenda da mensagem. O bloco de 512 bits é dividido em 16 palavras de 32 bits (W0 a W15), escritas em hexadecimal. No exemplo, W0 = 52656442 (“RedB”), W1 = 6c6f636b (“lock”), W2 = 426c7565 (“Blue”), W3 = 80000000 (o bit 1 do preenchimento) e W15 = 00000060 (o tamanho, 96). Depois, essas 16 palavras são expandidas para 64 pela fórmula $W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$, em que σ0 e σ1 combinam rotações (ROTR), deslocamentos (SHR) e XOR, e a soma é feita módulo 2^{32}. Por exemplo, W16 = σ0(W1) + W0 = 038fe9b0 + 52656442 = 55f54df2.

Passo 5 – Compressão (64 rodadas). Oito variáveis de trabalho (a até h) começam com os valores iniciais H0 a H7. Em cada uma das 64 rodadas, são calculados dois valores temporários: T1 = h + Σ1(e) + Ch(e, f, g) + Kt + Wt e T2 = Σ0(a) + Maj(a, b, c). A função Ch (“choose”) usa os bits de e para escolher, bit a bit, entre f e g, e a função Maj (“maioria”) devolve, em cada posição, o bit que aparece em pelo menos dois de a, b e c; Σ0 e Σ1 combinam três rotações com XOR para espalhar os bits. Em seguida, as variáveis são deslocadas: h recebe g, g recebe f, f recebe e, e recebe d + T1, d recebe c, c recebe b, b recebe a e a recebe T1 + T2. Na primeira rodada do exemplo, T1 = 45dd51aa e T2 = 08909ae5, o que resulta em a = 4e6dec8f e e = eb2d46e4.

Passo 6 – Atualização do hash. Após as 64 rodadas, cada variável é somada (módulo 2^{32}) ao valor inicial correspondente: H0 = H0 + a, H1 = H1 + b, e assim por diante até H7 = H7 + h. No exemplo, H0 = 6a09e667 + 8ee67912 = f8f05f79. Se a mensagem tivesse mais de um bloco, os passos 4 a 6 seriam repetidos para cada bloco, partindo desses novos valores.

Passo 7 – Resultado. O hash final é a concatenação de H0 a H7, totalizando 8 palavras de 32 bits, ou seja, 256 bits. Para “RedBlockBlue”, o resultado é f8f05f79 fe0c0f87 6d26368b d12c08ef 31617039 ae3104c3 4f22db9c 0afd3bd9.

2. Principais algoritmos de hash utilizados atualmente

Uma função de hash transforma qualquer dado em um valor de tamanho fixo. Ela é usada para garantir a segurança de dados, como na verificação da integridade de arquivos, em assinaturas digitais e no armazenamento de senhas.

Um hash seguro precisa resistir a colisões e ser de mão única, ou seja, não deve ser possível recuperar a mensagem original a partir do hash.

O MD5 foi criado em 1992, gera hashes de 128 bits e, por ser muito rápido, foi amplamente usado. Porém, hoje é possível gerar colisões nele em segundos, e malwares já se aproveitaram disso: o Flame, em 2012, usou uma colisão MD5 para forjar um certificado da Microsoft. O SHA-1, de 1995, com 160 bits, foi seu sucessor e foi usado por décadas. Em 2017, o Google e o CWI criaram dois PDFs diferentes com o mesmo hash SHA-1, e em 2022 o NIST determinou que ele seja abandonado até 2030.

Atualmente, o padrão é o SHA-2, usado em diversas aplicações, como TLS, certificados digitais e Bitcoin, e sem ataque prático conhecido até hoje. Seu ponto fraco é o ataque de extensão de comprimento (length extension): quem conhece o hash de uma mensagem que contém um segredo consegue estendê-la e calcular o novo hash sem conhecer o segredo, por isso se usa o HMAC para autenticação. O SHA-3 usa outra estrutura, imune a esse ataque, mas é mais lento e menos difundido. Há também o BLAKE2 e o BLAKE3, muito rápidos e seguros, mas que não são padrões do NIST.

Para senhas, um hash rápido é ruim, pois uma GPU consegue testar bilhões de senhas por segundo em um ataque de força bruta. Por isso, devem ser usados hashes lentos de propósito e com salt, como o bcrypt e o Argon2. Conclui-se que não existe um hash melhor para tudo: a escolha sempre depende do uso, e o MD5 e o SHA-1 não devem mais ser usados para fins de segurança.

3. Hash simples: XOR com rotação

Hash de 32 bits que processa a mensagem bloco a bloco: acrescenta o byte 0x80 e zeros até um múltiplo de 4 bytes, divide em blocos de 32 bits e, a partir do valor inicial 6A09E667, faz h = rotl(h, 5) XOR bloco para cada bloco. A rotação faz a ordem dos blocos importar, mas a função continua linear: é fácil montar colisões (“DCC075!!” e “wxAsPUiG” têm o mesmo hash, 989B5033). O código completo, com a tela e os testes, está no repositório: github.com/lucasdu01/UFJF-DCC075-Seguranca (pacote `hash`).

```
package br.ufjf.dcc075.hash;

import org.springframework.stereotype.Service;

import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HexFormat;
import java.util.List;

/**
 * Hash simples de 32 bits baseado em XOR com rotação, bloco a bloco.
 * <p>
 * 1. Padding: adiciona o byte 0x80 e completa com zeros até um múltiplo de 4 bytes
 *    (mesma ideia do padding do SHA: "a" e "a\0" não viram a mesma entrada).
 * 2. Divide a mensagem em blocos de 32 bits.
 * 3. Para cada bloco: h = rotl(h, 5) XOR bloco, começando de um valor inicial fixo.
 * <p>
 * A rotação faz a ordem dos blocos importar (só com XOR, "abcdefgh" e "efghabcd"
 * dariam o mesmo hash). Mas a função continua LINEAR: não é segura (ver README).
 */
@Service
public class HashXorRotacao {

    /** Valor inicial: os mesmos 32 bits iniciais (H0) do SHA-256. */
    static final int VALOR_INICIAL = 0x6A09E667;

    /** Quantos bits o estado gira para a esquerda antes de cada XOR. */
    static final int ROTACAO = 5;

    private static final HexFormat HEX = HexFormat.of().withUpperCase();

    /** Um passo do cálculo: o bloco processado e o estado h logo depois (para exibir na tela). */
    public record Passo(String bloco, String estado) {}

    /** Hash de um texto (UTF-8) em 8 dígitos hexadecimais. */
```

```
    public String calcularHex(String texto) {
        return HEX.toHexDigits(calcular(texto.getBytes(StandardCharsets.UTF_8)));
    }

    /** Hash de 32 bits de uma sequência de bytes. */
    public int calcular(byte[] mensagem) {
        int h = VALOR_INICIAL;
        for (int bloco : blocos(mensagem)) {
            h = Integer.rotateLeft(h, ROTACAO) ^ bloco; // bits que saem pela esquerda voltam pela
direita
        }
        return h;
    }

    /** Mesmo cálculo de {@link #calcular}, guardando cada passo para mostrar na tela. */
    public List<Passo> passos(String texto) {
        List<Passo> passos = new ArrayList<>();
        int h = VALOR_INICIAL;
        for (int bloco : blocos(texto.getBytes(StandardCharsets.UTF_8))) {
            h = Integer.rotateLeft(h, ROTACAO) ^ bloco;
            passos.add(new Passo(HEX.toHexDigits(bloco), HEX.toHexDigits(h)));
        }
        return passos;
    }

    /** Padding (0x80 + zeros até múltiplo de 4 bytes) e divisão em blocos de 32 bits. */
    int[] blocos(byte[] mensagem) {
        int tamanho = (mensagem.length + 1 + 3) / 4 * 4; // +1 do byte 0x80, arredondado para múltiplo de
4
        byte[] comPadding = Arrays.copyOf(mensagem, tamanho); // completa com zeros
        comPadding[mensagem.length] = (byte) 0x80;

        ByteBuffer buffer = ByteBuffer.wrap(comPadding); // lê de 4 em 4 bytes (big-endian)
        int[] blocos = new int[tamanho / 4];
        for (int i = 0; i < blocos.length; i++) {
            blocos[i] = buffer.getInt();
        }
        return blocos;
    }
}
```



Execução: hash de “Ola, mundo!” e o estado depois de cada bloco.

Fontes

[1] Vídeo indicado no enunciado: https://www.youtube.com/watch?v=orIgy2MjqrA

[2] NIST. *Secure Hash Standard (SHS)*. FIPS 180-4, 2015.

[3] NIST. *NIST Retires SHA-1 Cryptographic Algorithm*, 2022. https://www.nist.gov/news-events/news/2022/12/nist-retires-sha-1-cryptographic-algorithm